E-MAGAZINE

ISSUE 01 / OCTOBER 2026

UNDER THE HOOD

The systems that run fast, and the security that keeps them trusted.

SYSTEMS, INFRASTRUCTURE & PERFORMANCE

01 The Rust Migration

02 eBPF: Programmable Kernels

03 LSM-Trees vs B+ Trees

CYBERSECURITY & IDENTITY

04 Post-Quantum Cryptography

05 Deepfake Detection

06 Zero Trust Architecture

Six features / Systems, infrastructure, performance, cybersecurity and identity

EDITOR'S NOTE

Fast enough to matter. Safe enough to trust.

Every system we depend on is judged twice: once on how quickly it works, and once on whether we can believe it. This issue puts those two questions side by side.

In **Systems, Infrastructure & Performance** we look at three engineering shifts happening below the surface: the migration of kernels and web tooling to Rust, the eBPF virtual machine that lets you reprogram a running Linux kernel safely, and the storage-engine trade-off between B+ trees and LSM-trees that you first met in your DBMS course and will meet again in every production database.

In **Cybersecurity & Identity** we look at what happens when trust itself is under attack: cryptography facing a quantum computer, synthetic voices and faces that fool people, and the security model that stopped assuming the inside of the network is friendly.

The two halves share one idea. Memory-safe languages, verified kernel programs, quantum-safe keys and short-lived credentials all do the same thing: they move a guarantee out of human discipline and into the machinery. Read on.

IN THIS ISSUE

SYSTEMS, INFRASTRUCTURE & PERFORMANCE

01 **The Rust Migration: Rewriting the Modern Systems Stack** 4
Memory safety without garbage collection, from Linux and Windows to Turbopack and Biome.

02 **eBPF: Programmable Linux Kernels Without Modifying Source Code** 6
Real-time observability, modern firewalling and low-overhead profiling.

03 **Database Internals: LSM-Trees vs B+ Trees in Real-World Workloads** 8
From DBMS theory to RocksDB, Cassandra and the traditional SQL engines.

CYBERSECURITY & IDENTITY

04 **Post-Quantum Cryptography: Preparing for “Q-Day”** 11
NIST's finalized standards and how RSA and ECC will be replaced.

05 **Deepfake Detection: Audio Cloning & Multi-Modal Synthetic Media** 13
Spectral and phase analysis, watermarking and C2PA provenance.

06 **Zero Trust Architecture in Modern Cloud Deployments** 15
Continuous verification, micro-segmentation and ephemeral credentials.

SECTION 01

Systems, Infrastructure & Performance

Three shifts happening below the surface: safer languages, programmable kernels, and the storage engines that decide how fast data moves.

01 **The Rust Migration** p. 4
Memory safety without garbage collection

02 **eBPF: Programmable Linux Kernels** p. 6
Observability, firewalling and profiling

03 **LSM-Trees vs B+ Trees** p. 8
Database internals in real workloads

SECTION 01 / SYSTEMS, INFRASTRUCTURE & PERFORMANCE / FEATURE

The Rust Migration: Rewriting the Modern Systems Stack

Memory safety without a garbage collector sounds like a contradiction. It is the reason Linux, Windows and the tools behind the modern web keep moving to Rust.

E-MAGAZINE DESK / 7 MIN READ

For fifty years, systems programmers have faced a bargain. Write in C or C++ and you get raw speed and full control, plus a steady stream of buffer overflows, use-after-free bugs and data races. Write in Java, Go or C# and a garbage collector protects you, but you pay in pauses, memory overhead and unpredictable latency that a kernel or a packet-processing loop cannot afford. Rust's pitch is that the bargain was a false choice.

SAFETY CHECKED AT COMPILE TIME

Rust enforces three rules through its **ownership** model. Every value has exactly one owner. You may hold many shared, read-only references or one exclusive, mutable reference, never both at once. And no reference may outlive the value it points to. The compiler verifies this before the program runs, so entire bug classes (dangling pointers, double frees, iterator invalidation, data races between threads) simply fail to compile.

There is no collector at runtime. Memory is released deterministically at the moment its owner goes out of scope, which is why Rust can live inside a kernel driver.

```
let s = String::from("hello");
let t = s;          // ownership moves to t
println!("{}", s);  // compile error: borrow
                    // of moved value `s`
```

> *"Around 70% of our serious security bugs are memory-safety problems."*
>
> Finding reported independently by Microsoft (MSRC) and the Chromium project

LINUX: THE EXPERIMENT IS OVER

Rust entered the Linux kernel in version 6.1 (late 2022) as an experiment. At the 2025 Kernel Maintainers Summit developers agreed the experiment had succeeded, and a patch removing the "experimental" label was sent during the Linux 7.0 merge window in February 2026. Rust is already in production: Android 16 devices on the 6.12 kernel ship the anonymous shared memory allocator (ashmem) rewritten in Rust, and new drivers such as the open Nova driver for NVIDIA GPUs are being written in it.

The policy is deliberately incremental. Rust is for new drivers and subsystems; there is no plan to rewrite the existing C. Even so, the C interfaces have become cleaner because Rust bindings forced maintainers to state their rules (who owns this, who frees it) explicitly.

WINDOWS AND THE WIDER STACK

Microsoft has shipped Rust inside the Windows kernel, in parts of the graphics and windowing stack, and its Azure leadership has publicly said new projects should prefer Rust over C and C++. Google uses it across Android, parts of Chromium and Fuchsia. Debian made Rust a hard requirement for its APT package manager from May 2026, a sign that the language is moving from "optional" to "assumed" in the distribution layer.

RUST AT A GLANCE

- **Linux:** Rust since 6.1; experiment concluded Dec 2025, formalized in 7.0.
- **Android 16:** ashmem allocator written in Rust.
- **Next.js 16:** Turbopack is the default bundler.
- **Biome 2:** linter and formatter with type-aware rules, no TypeScript compiler needed.
- **Debian APT:** Rust required from May 2026.

THE WEB TOOLCHAIN: SPEED AS A SIDE EFFECT

Most developers meet Rust without writing it. **Turbopack**, the Rust bundler from Vercel, became the default for both dev and production builds in Next.js 16. **Biome** replaces ESLint and Prettier with one binary that parses each file once; version 2 added type-aware linting. Vite 8 now bundles with Rolldown, also written in Rust, and SWC and Oxc handle transpiling and linting elsewhere.

The speed comes from three things the language makes cheap: no garbage-collector pauses, true multi-threading without data races, and a single shared syntax tree instead of every tool re-parsing the same file. The trade-off is real. Biome does not yet have an equivalent for every ESLint plugin, so many teams run both during migration.

THE HONEST CAVEATS

Rust has a steep learning curve and slow compile times. Code marked `unsafe` still exists and must be audited, and the boundary where Rust calls C is exactly where bugs hide. Rust does not stop logic errors, only memory errors. That is why successful migrations are gradual: new code in Rust, old code left alone, with interfaces between them kept small.

KEY TAKEAWAY

You do not need to rewrite anything to benefit. Learn ownership and borrowing first, then read a small Rust kernel module or CLI tool. The concepts transfer to how you think about resources in every other language.

SECTION 01 / SYSTEMS, INFRASTRUCTURE & PERFORMANCE / FEATURE

eBPF: Programmable Linux Kernels Without Modifying Source Code

A sandboxed virtual machine inside the kernel lets you observe, filter and reshape traffic in real time, with no kernel patch, no module and no reboot.

E-MAGAZINE DESK / 7 MIN READ

Changing what the Linux kernel does used to mean one of two things. You could submit a patch and wait years for every distribution to ship it, or you could load a kernel module and hope it never crashed the machine. **eBPF** (extended Berkeley Packet Filter) is a third path: small programs, checked for safety, loaded into a running kernel on demand.

FROM PACKET FILTER TO KERNEL VM

BPF began in 1992 as a compact way for tools like tcpdump to filter packets. The extended version, merged into Linux in the 3.18 release (2014), is a general-purpose virtual machine inside the kernel with its own instruction set, 64-bit registers and just-in-time (JIT) compiler. Programs are event-driven: they run when something happens, such as a system call, a function entry, a timer or a packet arriving at the network card.

HOW A PROGRAM GETS INTO THE KERNEL

You write restricted C (or Rust), compile it to eBPF bytecode and hand it to the kernel through the `bpf()` system call. Before anything runs, the **verifier** analyses every possible execution path. It rejects programs that might read invalid memory, use uninitialized values, call unapproved helpers or fail to terminate. Only then is the program JIT-compiled and attached to a hook.

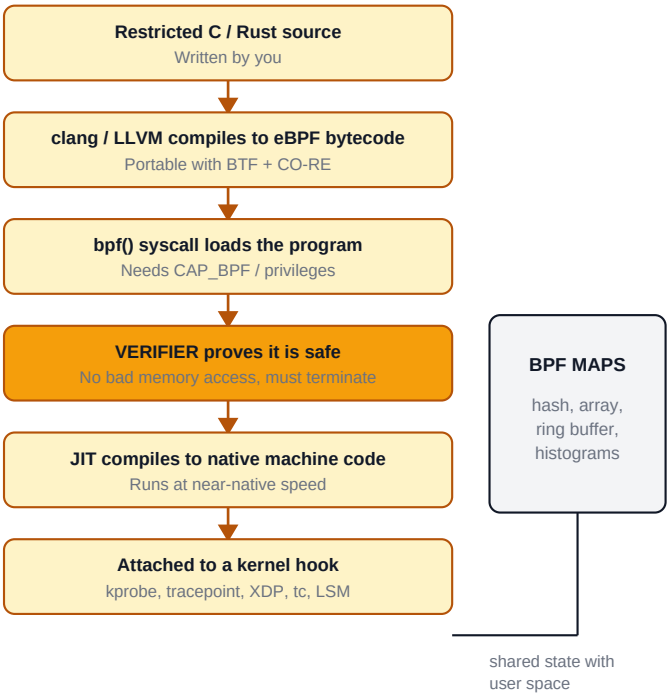


Figure 1. Life of an eBPF program. The verifier is the safety gate that replaces trust.

Programs keep state and talk to user space through **maps**: hash tables, arrays, ring buffers and histograms shared between the kernel program and a user-space controller.

OBSERVABILITY WITH ALMOST NO OVERHEAD

Traditional tracing copies every event to user space for analysis. eBPF aggregates inside the kernel, so only the summary (a histogram, a count per process) crosses the boundary. That is why you can leave it running in production. This one-liner for **bpftrace** prints every file opened on the machine:

```
bpftrace -e '
tracepoint:syscalls:sys_enter_openat {
  printf("%s opened %s\n", comm,
         str(args->filename));
}'
```

The same mechanism powers tools such as BCC, continuous profilers like Parca, and the kernel-level tracing behind Kubernetes observability platforms.

> *“Instead of moving all the data to the analysis, eBPF moves the analysis to the data.”*
>
> The core design idea behind eBPF observability

NETWORKING AND FIREWALLING

XDP (eXpress Data Path) attaches an eBPF program at the network driver, before the kernel allocates its normal packet structures. A program can drop, redirect or rewrite a packet at that earliest point, which is why it is used for DDoS mitigation at companies such as Cloudflare and for Meta’s Katran load balancer.

Cilium uses eBPF to replace iptables and kube-proxy in Kubernetes. Instead of walking a long linear chain of rules for every packet, it performs hash-map lookups that take roughly constant time, and it enforces policy by workload identity rather than by IP address. Security tools such as Falco and Tetragon use the same hooks to detect suspicious system calls and, with BPF-LSM, to block them.

WHERE EBPF HOOKS IN

- **kprobes / uprobes:** entry and exit of kernel or user functions.
- **Tracepoints:** stable, named kernel events.
- **XDP:** earliest packet hook, at the network driver.
- **tc:** traffic control, ingress and egress.
- **cgroup and LSM hooks:** per-container policy and security decisions.

LIMITS AND RISKS

The verifier caps program complexity, so some algorithms must be split up. Loading programs needs elevated privileges (CAP_BPF or more), and the same power has been abused to build rootkits, so production systems restrict who may load programs. Kernel internals differ between versions; **BTF** and **CO-RE** (compile once, run everywhere) let a single compiled program adapt to them.

KEY TAKEAWAY

eBPF turns the kernel from a fixed product into a platform. If you work with Linux, containers or networking, start with bpftrace one-liners and read what the verifier rejects; that is where the model clicks.

SECTION 01 / SYSTEMS, INFRASTRUCTURE & PERFORMANCE / FEATURE

Database Internals: LSM-Trees vs B+ Trees in Real-World Workloads

The DBMS syllabus meets production. Two families of storage engine make opposite bets about reads, writes and disks, and the bets decide which database fits your workload.

E-MAGAZINE DESK / 8 MIN READ

Every database answers the same question: where on disk do the bytes go? In your DBMS course the answer was a B+ tree. In the engines behind RocksDB, Cassandra and many time-series and messaging systems, the answer is a **log-structured merge-tree** (LSM-tree). Neither is better. They optimize different things.

THE B+ TREE: READ-OPTIMIZED, UPDATE IN PLACE

A B+ tree is a balanced tree of fixed-size pages (commonly 4 to 16 KB). Internal nodes hold only keys; leaves hold the records and are linked together so range scans just follow the chain. With fan-out in the hundreds, three or four levels cover billions of rows, so a lookup costs $O(\log_f N)$ page reads, usually a handful. Updates modify pages in place, protected by a write-ahead log. This is the engine inside InnoDB (MySQL), PostgreSQL, SQLite, Oracle and SQL Server.

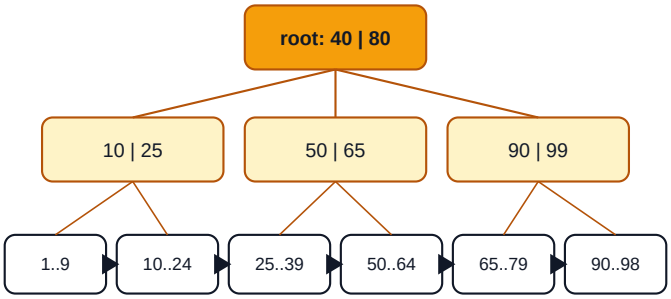
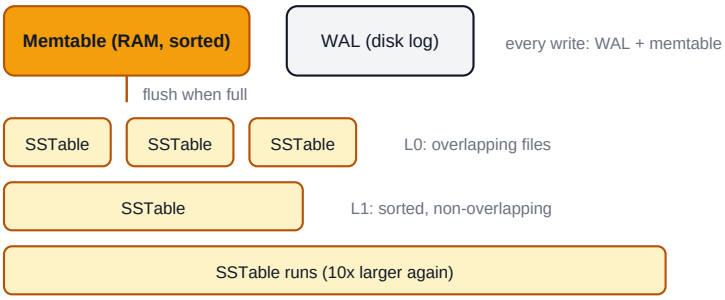


Figure 1. B+ tree: one root-to-leaf path per lookup; linked leaves serve range scans.

The cost shows up on writes. A one-byte change rewrites a whole page, random page writes scatter across the disk, and a full page must **split**, touching its parent as well.

THE LSM-TREE: WRITE-OPTIMIZED, APPEND AND MERGE

An LSM-tree never updates in place. A write goes to a write-ahead log and into a sorted in-memory **memtable**. When the memtable fills it is flushed to disk as an immutable, sorted **SSTable**. Background **compaction** merges these files into larger levels and discards overwritten or deleted entries. Every disk write is sequential, which suits both spinning disks and SSDs.



Compaction merges runs downward: data is rewritten, but only sequentially (keeping reads from searching too many files)

Figure 2. LSM-tree: fast sequential ingestion; compaction does the tidying later.

A read must check the memtable and then files from newest to oldest. **Bloom filters** let the engine skip files that definitely do not contain the key, which keeps point lookups fast. Deletes are written as **tombstones** and only removed when compaction reaches them.

THE THREE AMPLIFICATIONS

	B+ tree	LSM-tree
Write amp.	Page rewrites plus WAL; random I/O	Compaction rewrites data many times, but sequentially
Read amp.	Low: one root-to-leaf path	Higher: several levels; Bloom filters help
Space amp.	Partly filled pages after splits	Stale versions until compaction

	B+ tree	LSM-tree
Range scans	Excellent, linked leaves	Good, but merges many runs
Best for	Read-heavy OLTP, joins, point and range queries	Write-heavy ingest: logs, metrics, messaging, wide-column stores

Table 1. Trade-offs at a glance. Tuning compaction moves an LSM along this curve.

WHAT REAL SYSTEMS CHOOSE

Cassandra, **ScyllaDB**, **HBase** and **RocksDB** use LSM-trees because their workloads are dominated by writes. Meta's MyRocks puts RocksDB underneath MySQL to trade some read speed for much better storage efficiency than InnoDB. **CockroachDB** serves SQL on top of Pebble, an LSM engine, showing that the storage engine and the query language are separate decisions.

Compaction strategy is the main tuning knob. **Leveled** compaction keeps reads and space tight but rewrites more data. **Size-tiered** compaction writes less but needs more free space and scans more files. **Time-window** strategies suit time-series data that expires in order.

A RULE OF THUMB

- Mostly reads, rich queries, transactions: **B+ tree**.
- Firehose of writes, key lookups, huge volumes: **LSM-tree**.
- Unsure: measure with your real data. Both families keep getting hybrid features.

KEY TAKEAWAY

When an exam asks for B+ tree insertion, you are learning the in-place world. When a production system struggles with write throughput or SSD wear, ask whether its engine is rewriting pages in place or merging runs in the background.

SECTION 02

Cybersecurity & Identity

When trust itself is the target: cryptography facing quantum computers, synthetic media, and a security model without a trusted inside.

04 **Post-Quantum Cryptography** p. 11
Preparing for Q-Day

05 **Deepfake Detection** p. 13
Audio cloning and multi-modal synthetic media

06 **Zero Trust Architecture** p. 15
Cloud security without a perimeter

SECTION 02 / CYBERSECURITY & IDENTITY / FEATURE

Post-Quantum Cryptography: Preparing for “Q-Day”

RSA and elliptic curves protect almost every secure connection today. A large quantum computer would break both, and the migration has already begun.

E-MAGAZINE DESK / 8 MIN READ

Nobody knows when a quantum computer capable of breaking today’s public-key cryptography will exist. Security engineers are moving anyway, because the data being encrypted now may still need to be secret when that day comes. The day itself has a name: **Q-Day**.

WHAT QUANTUM ACTUALLY BREAKS

Shor’s algorithm (1994) factors large integers and computes discrete logarithms in polynomial time on a large, fault-tolerant quantum computer. Those two problems are the hard problems behind RSA, Diffie-Hellman, ECDH and ECDSA. Symmetric cryptography fares much better: Grover’s algorithm gives only a quadratic speed-up, so AES-256 and SHA-384 stay comfortable.

The urgent threat is **harvest now, decrypt later**: an adversary records encrypted traffic today and decrypts it once a quantum computer is available. Anything that must stay confidential for ten or twenty years (health records, identity data, state secrets) is already exposed in principle.

THE NEW STANDARDS

After an eight-year public competition, NIST published its first three post-quantum standards on 13 August 2024.

Standard	Algorithm	Job	Basis
FIPS 203	ML-KEM (from Kyber)	Key establishment	Module lattices
FIPS 204	ML-DSA (from Dilithium)	Digital signatures	Module lattices
FIPS 205	SLH-DSA (from SPHINCS+)	Signatures (backup)	Hash functions

Table 1. NIST’s finalized post-quantum standards.

NIST added **HQC**, a code-based scheme, in March 2025 as a second key-encapsulation option, so a break of lattices would not leave the world with a single choice, and FN-DSA (based on Falcon) is still in the standardization pipeline. The lattice idea behind ML-KEM and ML-DSA is *learning with errors*: recovering a secret from many noisy linear equations appears to be hard even for quantum computers. SLH-DSA is deliberately conservative; its security depends only on hash functions.

> *“The standards are ready. Organizations do not need to wait for 2030 or 2035 to begin adopting them.”*
>
> NIST guidance, as summarized in its transition roadmap

WHAT CHANGES IN PRACTICE: SIZE

Post-quantum keys and signatures are much larger than their elliptic-curve counterparts. The computation is fast; the bytes are the problem.

Item	Classical	Post-quantum
Key exchange public key	X25519: 32 bytes	ML-KEM-768: 1,184 bytes
Key exchange ciphertext	32 bytes	ML-KEM-768: 1,088 bytes
Signature	ECDSA P-256: 64 bytes	ML-DSA-65: 3,309 bytes
Signature public key	64 bytes	ML-DSA-65: 1,952 bytes

Table 2. Approximate sizes. TLS handshakes and certificate chains grow accordingly.

Bigger handshakes strain constrained devices, DNS-based protocols and any system with tight packet limits. They are one reason the migration is not a simple drop-in swap.

THE HYBRID TRANSITION

Chrome and Firefox already support **hybrid key exchange**, combining X25519 with ML-KEM-768 so that a connection

stays safe as long as either algorithm holds. Key exchange moves first because of harvest-now-decrypt-later. Signatures and certificates can follow, since forging one requires a quantum computer at the moment of attack, not years later.

THE TIMELINE

NIST's transition roadmap (IR 8547) proposes deprecating RSA and elliptic-curve cryptography at the 112-bit security level after 2030 and disallowing it after 2035. The practical advice is the same everywhere: inventory where you use public-key cryptography, protect long-lived data first, and build **crypto-agility** so that algorithms can be swapped without a rebuild.

TRY IT YOURSELF

- OpenSSL 3.5 added ML-KEM, ML-DSA and SLH-DSA support.
- Check which key exchange your browser negotiates in the developer tools security tab.
- List every place your project uses RSA or ECDSA. That is your migration inventory.

KEY TAKEAWAY

Q-Day is a deadline with an unknown date, so the rational plan is to start with the data that must outlive the threat. Standards exist, browsers already use them, and the main work left is engineering.

SECTION 02 / CYBERSECURITY & IDENTITY / FEATURE

Deepfake Detection: Audio Cloning and Multi-Modal Synthetic Media

Generative models improve every quarter. Detectors, watermarks and provenance standards are racing to keep up, and none of them wins alone.

E-MAGAZINE DESK / 7 MIN READ

A short voice sample is now enough for modern models to produce a convincing clone. Fraudsters use cloned voices to impersonate relatives and executives; fake video calls and manipulated clips follow the same pattern. Telling real from synthetic has become a technical contest between generators and detectors.

AUDIO: WHERE CLONES LEAK

A neural vocoder builds speech as a waveform from learned features. It sounds natural to people, but it leaves measurable traces, and detectors look for them in several domains.

Spectral analysis. Audio is converted to a time-frequency representation such as a mel-spectrogram or constant-Q features. Synthetic speech often shows unusually smooth high-frequency energy, missing breath and room-noise micro-variation, or periodic patterns introduced by upsampling layers.

Phase artifacts. Many vocoders optimize the magnitude of the spectrum and reconstruct phase approximately. Features that preserve phase, such as group delay or instantaneous frequency, can expose inconsistencies that magnitude-only features hide.

Learned detectors. A common architecture pairs a self-supervised front end such as wav2vec 2.0 with a classifier back end such as AASIST, which applies graph attention over spectral and temporal features. The ASVspoof challenges provide shared benchmarks for this work.

> *“Detection tells you what a file looks like. Provenance tells you where it came from. You need both.”*
>
> The layered-defense view of synthetic media

VIDEO AND MULTI-MODAL SIGNALS

Image and video generators leave different fingerprints: periodic peaks in the frequency domain from upsampling, blending seams at face boundaries, inconsistent lighting or eye reflections, and flicker between frames. Biological signals such as the faint colour pulse of blood flow in facial skin (remote photoplethysmography) are hard for generators to reproduce faithfully.

Multi-modal detectors look across channels. A lip-sync model embeds audio and mouth movement in a shared space and flags clips where they disagree. A forger must now fake the voice, the face and the alignment between them.

THE CAT-AND-MOUSE PROBLEM

Detectors trained on known generators often degrade on new ones. Compression, resampling and phone-call codecs destroy delicate artifacts. Attackers can add adversarial noise or fine-tune a generator against a public detector. Detection output is therefore a probability, not proof, and it must be treated that way.

WATERMARKS AND PROVENANCE

The complementary approach marks content at creation. **Invisible watermarks**, such as Google’s SynthID for images and audio or Meta’s AudioSeal for speech, embed a signal in the media itself that survives some edits. **C2PA Content Credentials** attach a cryptographically signed manifest that records who created the file and how it was edited. The manifest is bound to the content by a hash and signed with certificates, so tampering is detectable.

Approach	Strength	Weakness
Detection	Works on any file, no cooperation needed	Probabilistic; fails on unseen generators

Approach	Strength	Weakness
Watermark	Survives re-encoding; machine-readable	Only covers cooperating generators
C2PA provenance	Tamper-evident, signed history	Metadata can be stripped; absence proves nothing

Table 1. Three defenses with complementary gaps.

DEFENSE IN LAYERS

Real deployments combine all three, then add process. Banks and families alike benefit from out-of-band verification: call back on a known number, or agree a private code word for urgent money requests. A convincing voice is no longer evidence of identity.

HOW TO EVALUATE A DETECTOR

- Test on generators it was **not** trained on.
- Test after compression and resampling, not just on clean files.
- Report equal error rate, and read the false-accept rate, not only accuracy.
- Never use a single score as proof; use it to triage.

KEY TAKEAWAY

No single technique defeats synthetic media. Combine spectral and phase-aware detection, watermarks, signed provenance and human verification, and assume the generators will keep improving.

SECTION 02 / CYBERSECURITY & IDENTITY / FEATURE

Zero Trust Architecture in Modern Cloud Deployments

The castle-and-moat model assumed everything inside the network was friendly. Zero Trust assumes nothing is, and checks every request.

E-MAGAZINE DESK / 7 MIN READ

For decades, security meant a strong perimeter: firewalls at the edge and a trusted network inside. Cloud services, remote work, SaaS tools and containers dissolved that edge. Once an attacker held one stolen password, a flat internal network let them move sideways to everything else.

WHY THE MOAT FAILED

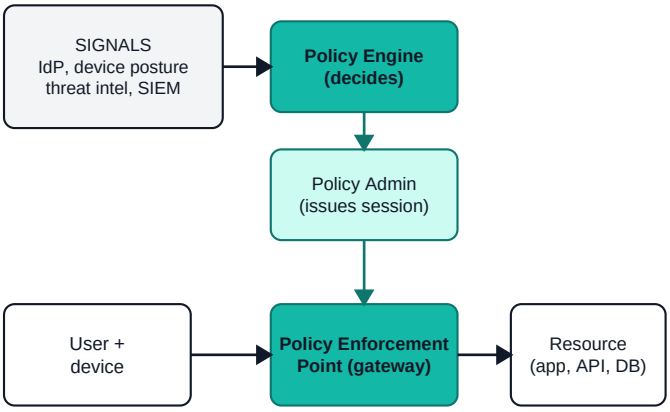
Google's **BeyondCorp** project, started after a major intrusion in 2009, was an early large-scale answer: treat every network as hostile and grant access based on who is asking and from what device. NIST later codified the approach in SP 800-207, the Zero Trust Architecture standard.

THREE PRINCIPLES

Verify explicitly. Decide on every request using identity, device health, location and behaviour, not network position. **Use least privilege.** Give just enough access, for just enough time. **Assume breach.** Design so that a compromised component can reach as little as possible, and log everything so you can see it happen.

THE MOVING PARTS

NIST describes three logical components. A **Policy Engine** decides whether a request is allowed. A **Policy Administrator** carries out that decision, for example by issuing a session token. A **Policy Enforcement Point**, usually a gateway or proxy, sits in the path and blocks or permits traffic. The engine draws on signals from the identity provider, device-posture checks, threat intelligence and the security-monitoring system.



Every request passes the PEP; the engine re-evaluates trust each time.
No request is trusted because of the network it came from.

Figure 1. NIST-style Zero Trust components. The decision is made per request.

> *"Never trust, always verify" replaces "trust but verify, once, at the door."*
>
> The short version of Zero Trust

PILLAR 1: CONTINUOUS IDENTITY VERIFICATION

Identity becomes the new perimeter. That means single sign-on, phishing-resistant multi-factor authentication such as FIDO2 passkeys, and device attestation that proves the laptop is managed and patched. Trust is not granted once at login: sessions are re-evaluated, and a risky signal (a new location, an impossible travel pattern) triggers a step-up challenge or ends the session.

PILLAR 2: MICRO-SEGMENTATION

Instead of one large trusted network, each workload gets its own boundary. In cloud deployments that means narrow security groups, Kubernetes network policies and a service

mesh that enforces **mutual TLS** between services. Workloads authenticate by cryptographic identity (for example SPIFFE identities) rather than by IP address, so a compromised container cannot simply scan its neighbours.

PILLAR 3: EPHEMERAL CREDENTIALS

Long-lived passwords and API keys are the attacker's favourite loot. Zero Trust replaces them with short-lived credentials: cloud role tokens that expire in minutes, OIDC federation so CI pipelines never store cloud keys, and secrets managers that generate database credentials on demand. A stolen credential then ages out on its own.

	Castle and moat	Zero Trust
Trust basis	Network location	Identity, device and context
Access	Broad once inside	Per request, least privilege
Credentials	Long-lived passwords and keys	Short-lived, auto-rotated
Lateral movement	Easy on a flat network	Contained by segmentation
Visibility	Perimeter logs	Per-request telemetry

Table 1. Two security models compared.

PITFALLS

Zero Trust is a strategy, not a product you can buy. Legacy applications that assume a trusted network are hard to retrofit. Policy can sprawl into something nobody understands, and the identity provider becomes a critical single point of failure. Teams that succeed start with their most valuable systems, fix identity first, and migrate one application at a time.

WHERE TO START

- Enforce phishing-resistant MFA on every admin account.
- Remove long-lived cloud keys from code and CI; use federation.
- Map which services talk to which, then deny everything else.
- Turn on logging for every access decision before tightening rules.

KEY TAKEAWAY

Zero Trust changes the question from "is this request inside our network?" to "does this identity, on this device, need this resource right now?" Short-lived credentials and small boundaries make a breach smaller by design.

GLOSSARY

Terms from this issue

Ownership. Rust rule: each value has one owner, freed when the owner leaves scope.

Borrow checker. The Rust compiler pass that enforces reference rules.

eBPF verifier. Kernel component that proves a program is safe before it runs.

XDP. eXpress Data Path: eBPF hook at the network driver.

SSTable. Immutable, sorted file written by an LSM-tree.

Compaction. Background merge of SSTables that removes stale data.

Write amplification. Bytes written to disk per byte written by the application.

Bloom filter. Compact structure that says a key is absent or possibly present.

Q-Day. The day a quantum computer can break RSA and elliptic-curve cryptography.

ML-KEM. NIST FIPS 203 post-quantum key encapsulation mechanism.

ML-DSA. NIST FIPS 204 post-quantum signature scheme.

Harvest now, decrypt later. Recording encrypted traffic today to decrypt it in future.

C2PA. Open standard for signed content provenance (Content Credentials).

Vocoder. Neural model that turns acoustic features into a waveform.

Micro-segmentation. Giving each workload its own network and identity boundary.

Ephemeral credential. A credential that expires in minutes or hours.

SOURCES AND FURTHER READING

Where the facts came from

- Phoronix, “Linux 7.0 Officially Concluding The Rust Experiment” (9 Feb 2026), phoronix.com/news/linux-7.0-rust
- LWN.net, “The (successful) end of the kernel Rust experiment” (Dec 2025), lwn.net/Articles/1049948
- NIST FIPS 203, 204 and 205 (13 Aug 2024) and NIST IR 8547, the post-quantum transition roadmap (csrc.nist.gov)
- NIST SP 800-207, Zero Trust Architecture (csrc.nist.gov)
- C2PA specification and Content Credentials (c2pa.org)
- Next.js 16, Vite 8 and Biome 2 release notes (project websites)
- ebpf.io, the eBPF project documentation, and the bpftrace reference guide
- RocksDB wiki, “Compaction”; Cassandra documentation on compaction strategies
- Your DBMS textbook chapters on B+ trees and indexing, for the in-place baseline

AUTHORS AND CONTRIBUTORS

Yogiraj Kulkarni	Roll No. B-03
Arnav Nandanwar	Roll No. B-07
Soham Pawar	Roll No. B-08
Vrushabh Yadav	Roll No. B-32